TÀI LIỆU ĐẶC TẢ KỸ THUẬT VÀ TRIỂN KHAI TOÀN DIỆN (A-Z)

DỰ ÁN: FRESHBITE COFFEE & BAKERY (ĐÀ NẴNG)

HỆ THỐNG ĐẶT TRƯỚC VÀ NHẬN TẠI QUẦY (ORDER AHEAD & PICK-UP SYSTEM)

Áp dụng liên thông 3 môn học: SDN302 (Node.js/Express) – MMA301 (React Native Expo) – PRM393 (Flutter)

PHẦN 1: TỔNG QUAN HỆ THỐNG VÀ BỐI CẢNH ĐÀ NẴNG

1.1. Bối cảnh & Phạm vi Dự án (Domain Scope)

- **Thương hiệu:** FreshBite Coffee & Bakery – Chuỗi cà phê đặc sản & bánh thủ công phục vụ sinh viên và dân văn phòng tại TP. Đà Nẵng.
- **Phạm vi địa lý:** Hoạt động độc quyền tại **TP. Đà Nẵng** với 4 chi nhánh chiến lược:
- **Chi nhánh 1 (FPT City - Ngũ Hành Sơn):** Phục vụ sinh viên ĐH FPT, ĐH CNTT-TT Việt - Hàn (VKU), cư dân FPT City. *(Tạo ấn tượng cực mạnh với giảng viên FPT ĐN)*.
- **Chi nhánh 2 (Hải Châu - Trung tâm):** 154 Bạch Đằng, P. Hải Châu 1, Q. Hải Châu (Tuyến phố đi bộ ven sông Hàn).
- **Chi nhánh 3 (Sơn Trà - Biển):** 68 Võ Văn Kiệt, P. An Hải Đông, Q. Sơn Trà (Trục đường thẳng ra biển Mỹ Khê).
- **Chi nhánh 4 (Thanh Khê - Sinh viên):** 242 Điện Biên Phủ, P. Chính Gián, Q. Thanh Khê.
- **Mô hình vận hành: Order Ahead & Pick-up (100% Không dùng Shipper)**.
- Khách dùng Mobile App chọn chi nhánh gần nhất qua GPS/Google Maps.
- Đặt bánh/cà phê, chọn giờ hẹn ghé lấy (15p, 30p, 45p).
- Quán pha chế/nướng bánh $\rightarrow$ Bắn thông báo qua App $\rightarrow$ Khách ghé quầy quét mã nhận đồ.
- **Phân quyền chính xác 2 Roles:**
- `user`: Khách hàng sử dụng Mobile App (iOS / Android).
- `admin`: Quản lý / Barista tại quầy dùng Web Admin (EJS trên SDN302) hoặc màn hình Admin trên Mobile để đổi trạng thái đơn và quản lý menu.

PHẦN 2: THIẾT KẾ CƠ SỞ DỮ LIỆU TOÀN DIỆN (MONGODB SCHEMAS & SEED DATA)

2.1. Chi tiết các Schema (Mongoose Models)

1. `Branch` Schema (`models/Branch.js`)

Lưu trữ 4 chi nhánh thực tế tại Đà Nẵng:

```
const mongoose = require('mongoose');

const branchSchema = new mongoose.Schema({
  name: { type: String, required: true },
  address: { type: String, required: true },
  district: { type: String, required: true, enum: ['Ngũ Hành Sơn', 'Hải Châu', 'Sơn Trà', 'Thanh Khê'] },
  latitude: { type: Number, required: true },
  longitude: { type: Number, required: true },
  phone: { type: String, required: true },
  openingHours: { type: String, default: '06:30 - 22:30' },
  imageUrl: { type: String, required: true },
  isActive: { type: Boolean, default: true }
}, { timestamps: true });

module.exports = mongoose.model('Branch', branchSchema);
```

2. `User` Schema (`models/User.js`)

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  fullName: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true, trim: true },
  password: { type: String, required: true },
  phone: { type: String, required: true, trim: true },
  avatar: { type: String, default: 'https://images.unsplash.com/photo-1535713875002-d1d0cf377fde?w=200' },
  role: { type: String, enum: ['user', 'admin'], default: 'user' },
  favoriteProducts: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Product' }]
}, { timestamps: true });

module.exports = mongoose.model('User', userSchema);
```

3. `Category` Schema (`models/Category.js`)

```
const mongoose = require('mongoose');

const categorySchema = new mongoose.Schema({
  name: { type: String, required: true, unique: true },
  code: { type: String, required: true, unique: true }, // 'coffee', 'bakery', 'tea', 'specialty'
  description: String,
  icon: String
}, { timestamps: true });

module.exports = mongoose.model('Category', categorySchema);
```

4. `Product` Schema (`models/Product.js`)

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  categoryId: { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true },
  price: { type: Number, required: true, min: 0 },
  description: { type: String, required: true },
  imageUrl: { type: String, required: true },
  calories: { type: Number, default: 150 }, // Phục vụ Gemini AI tư vấn dinh dưỡng
  ingredients: [{ type: String }],          // "Cà phê Robusta Đắk Lắk", "Muối hồng", "Sữa đặc"
  isBestSeller: { type: Boolean, default: false },
  isAvailable: { type: Boolean, default: true }
}, { timestamps: true });

module.exports = mongoose.model('Product', productSchema);
```

5. `Order` Schema (`models/Order.js`)

```
const mongoose = require('mongoose');

const orderItemSchema = new mongoose.Schema({
  productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },
  name: String,
  quantity: { type: Number, required: true, min: 1 },
  price: { type: Number, required: true },
  sugarLevel: { type: String, enum: ['100%', '70%', '50%', '0%'], default: '100%' },
  iceLevel: { type: String, enum: ['100%', '50%', 'Nóng'], default: '100%' },
  note: String
});

const orderSchema = new mongoose.Schema({
  orderCode: { type: String, required: true, unique: true }, // Format: FB-10023
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  branchId: { type: mongoose.Schema.Types.ObjectId, ref: 'Branch', required: true },
  items: [orderItemSchema],
  subtotal: { type: Number, required: true },
  discount: { type: Number, default: 0 },
  totalAmount: { type: Number, required: true },
  pickupTime: { type: String, required: true }, // 'Sau 15 phút (08:45)', '09:30 AM'
  status: {
    type: String,
    enum: ['Received', 'Preparing', 'Ready for Pickup', 'Completed', 'Cancelled'],
    default: 'Received'
  },
  paymentMethod: { type: String, enum: ['CASH_AT_COUNTER', 'MOCK_MOMO'], default: 'CASH_AT_COUNTER' },
  paymentStatus: { type: String, enum: ['PENDING', 'PAID'], default: 'PENDING' }
}, { timestamps: true });

module.exports = mongoose.model('Order', orderSchema);
```

2.2. Dữ liệu Mẫu Khởi tạo (Seed Data Đà Nẵng)

Tạo file `seeds/seedData.js` chứa dữ liệu thật: - **4 Chi nhánh Đà Nẵng:** `javascript const branches = [{ name: "FreshBite FPT City (Ngũ Hành Sơn)", address: "Tòa F-Complex, Khu đô thị FPT City, P. Hòa Hải, Q. Ngũ Hành Sơn, Đà Nẵng", district: "Ngũ Hành Sơn", latitude: 15.969234, longitude: 108.261221, phone: "0236.730.0999", openingHours: "06:30 - 22:00", imageUrl: "https://images.unsplash.com/photo-1554118811-1e0d58224f24?w=800" }, {`

```
name: "FreshBite Bạch Đằng (Hải Châu)", address: "154 Bạch Đằng, P. Hải Châu 1, Q. Hải Châu, Đà Nẵng (Ven sông Hàn)", district: "Hải Châu", latitude: 16.067822, longitude: 108.220815, phone: "0236.388.8222", openingHours: "06:30 - 23:00", imageUrl: "https://images.unsplash.com/photo-1501339847302-ac426a4a7cbb?w=800" }, { name: "FreshBite Võ Văn Kiệt (Sơn Trà)", address: "68 Võ Văn Kiệt, P. An Hải Đông, Q. Sơn Trà, Đà Nẵng (Đường ra biển Mỹ Khê)", district: "Sơn Trà", latitude: 16.061044, longitude: 108.238055, phone: "0236.399.9111", openingHours: "06:30 - 22:30", imageUrl: "https://images.unsplash.com/photo-1495474472287-4d71bcdd2085?w=800" }, { name: "FreshBite Điện Biên Phủ (Thanh Khê)", address: "242 Điện Biên Phủ, P. Chính Gián, Q. Thanh Khê, Đà Nẵng", district: "Thanh Khê", latitude: 16.062011, longitude: 108.185012, phone: "0236.377.7333", openingHours: "07:00 - 22:00", imageUrl: "https://images.unsplash.com/photo-1559925393-8be0ec4767c8?w=800" } ];
```

- **Món ăn đặc sắc Đà Nẵng:**
- Cà phê Muối Đà Nẵng (32.000 VNĐ, 180 kcal)
- Cà phê Dừa Non Xay (42.000 VNĐ, 260 kcal)
- Cold Brew Cam Vàng Sơn Trà (45.000 VNĐ, 80 kcal)
- Trà Lài Hạt Sen Cố Đô (38.000 VNĐ, 120 kcal)
- Bánh Croissant Bơ Tỏi Trứng Muối (35.000 VNĐ, 310 kcal)
- Bánh Tiramisu Vị Cà Phê Cốt Dừa (45.000 VNĐ, 280 kcal)
- Bánh Biscotti Ăn Kiêng Yến Mạch Hạnh Nhân (28.000 VNĐ, 110 kcal)

PHẦN 3: ĐẶC TẢ API & QUY TẮC VALIDATION (FE & BE)

3.1. Bảng Quy tắc Kiểm tra Hợp lệ (Validation Rules)

Trường dữ liệu	Quy tắc Frontend	Quy tắc Backend (Express Validator / Regex)	Thông báo lỗi
Email	Không rỗng, chuẩn email	`^[\w-\.]+@([\w-]+\.)+[\w-]{2,4}$`	*"Email không đúng định dạng hợp lệ!"*
Password	Độ dài ≥ 6 ký tự	`isLength({ min: 6 })`	*"Mật khẩu phải chứa ít nhất 6 ký tự!"*
Phone	Bắt đầu bằng 0, đủ 10 số	`^0[3|5|7|8|9][0-9]{8}$`	*"Số điện thoại phải là số di động VN 10 chữ số!"*
Full Name	Không rỗng, 3 - 50 ký tự	`trim().isLength({ min: 3, max: 50 })`	*"Họ và tên từ 3 đến 50 ký tự!"*
Pickup Time	Bắt buộc chọn từ Dropdown	`isIn(['15_MINS', '30_MINS', '45_MINS', 'CUSTOM'])`	*"Vui lòng chọn thời gian ghé nhận món!"*
Branch ID	Bắt buộc chọn từ Map/List	`isMongoId()`	*"Chi nhánh nhận hàng không hợp lệ!"*
Cart Items	Mảng không rỗng, min 1 món	`isArray({ min: 1 })`, `quantity >= 1`	*"Giỏ hàng chưa có sản phẩm nào!"*

3.2. Danh sách Endpoints RESTful API Chuẩn Mực

Module Auth (`/api/auth`)

- `POST /api/auth/register`: Đăng ký tài khoản (Validate email, phone, hash password bằng bcrypt).
- `POST /api/auth/login`: Đăng nhập (Trả về JWT token hiệu lực 7 ngày và User Object).
- `GET /api/auth/me`: Lấy thông tin user hiện tại (Yêu cầu Header `Authorization: Bearer <token>`).
- `PUT /api/auth/profile`: Cập nhật thông tin cá nhân.
- `POST /api/auth/upload-avatar`: Upload ảnh avatar qua Multer (`multipart/form-data`).

Module Branches (`/api/branches`)

- `GET /api/branches`: Lấy danh sách 4 chi nhánh tại Đà Nẵng kèm tọa độ GPS.
- `GET /api/branches/:id`: Xem chi tiết 1 chi nhánh.

Module Products & Categories (`/api/products` , `/api/categories`)

- `GET /api/categories`: Danh sách danh mục.
- `GET /api/products`: Lấy danh sách món. Query params: `?categoryId=...&search=...&sort=price_asc`.
- `GET /api/products/:id`: Lấy chi tiết món ăn (Populate `categoryId`).
- `POST /api/products`: [Admin] Tạo món mới (Upload ảnh qua Multer).

- `PUT /api/products/:id`: [Admin] Cập nhật món.
- `DELETE /api/products/:id`: [Admin] Xóa món.

Module Orders (`/api/orders`)

- `POST /api/orders`: Tạo đơn đặt trước lấy tại quầy.
- Body: `{ branchId, items: [{ productId, quantity, sugarLevel, iceLevel, note }], pickupTime, paymentMethod }`
- Sinh tự động mã đơn `orderCode`: `FB-XXXX` ngẫu nhiên 5 chữ số.
- `GET /api/orders/my-orders`: Lấy lịch sử đơn hàng của user đang đăng nhập (Sắp xếp mới nhất trước).
- `GET /api/orders/:id`: Chi tiết đơn hàng (Populate `branchId`, `items.productId`).
- `GET /api/orders/admin/all`: [Admin] Lấy danh sách tất cả đơn hàng theo trạng thái.
- `PATCH /api/orders/:id/status`: [Admin] Cập nhật trạng thái đơn hàng (`Preparing` $\rightarrow$ `Ready for Pickup` $\rightarrow$ `Completed`).

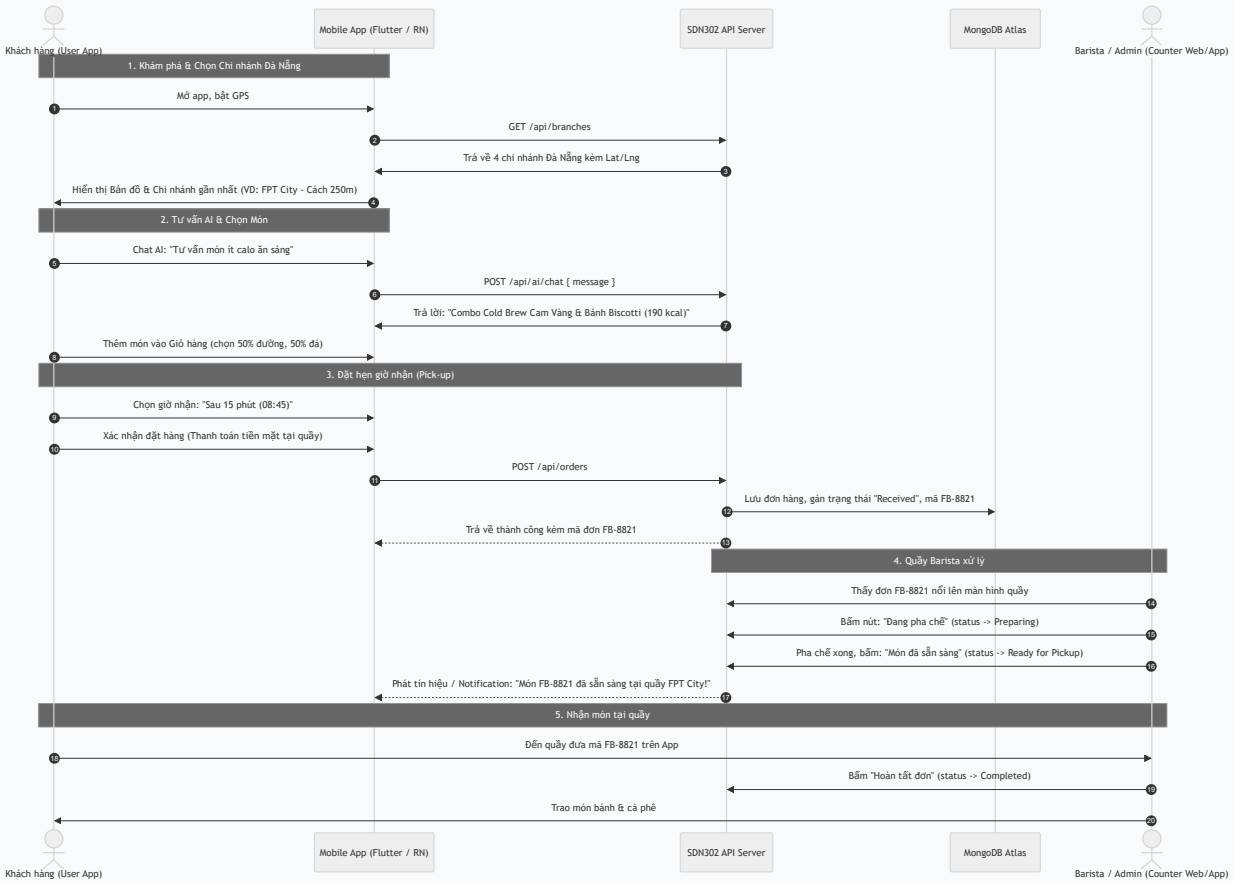
Module AI Assistant (`/api/ai`)

- `POST /api/ai/chat`: Nhận câu hỏi từ User $\rightarrow$ Gửi kèm System Prompt danh sách menu FreshBite Đà Nẵng $\rightarrow$ Trả về câu trả lời tư vấn dinh dưỡng và gợi ý món ăn.

Module Admin SSR Web (`/admin`)

- `GET /admin/dashboard`: Render trang EJS thống kê doanh thu và đơn hàng hôm nay.
- `GET /admin/orders`: Giao diện quầy Barista bấm nút chuyển trạng thái tức thì.
- `GET /admin/products`: Giao diện web quản lý thêm/sửa món ăn.

PHẦN 4: LUỒNG NGHIỆP VỤ TOÀN TRÌNH TỪ A-Z (USER & ADMIN)



PHẦN 5: BẢN HOÀN THIỆN MÔN 1 – SDN302 (NODE.JS + EXPRESS + MONGODB)

5.1. Cấu trúc Source Code Hoàn Chỉnh

```
sdn302-freshbite-backend/
├── config/
│   ├── db.js
│   └── gemini.js
├── controllers/
│   ├── authController.js
│   ├── branchController.js
│   ├── categoryController.js
│   ├── productController.js
│   ├── orderController.js
│   └── aiController.js
├── middlewares/
│   ├── authMiddleware.js
│   ├── uploadMiddleware.js
│   └── errorMiddleware.js
├── models/
│   ├── User.js
│   ├── Branch.js
│   ├── Category.js
│   ├── Product.js
│   └── Order.js
├── routes/
│   ├── authRoutes.js
│   ├── branchRoutes.js
│   ├── categoryRoutes.js
│   ├── productRoutes.js
│   ├── orderRoutes.js
│   └── aiRoutes.js
├── views/                    # Slot 7-8: EJS Admin Template
│   ├── layouts/
│   │   └── main.ejs
│   ├── dashboard.ejs
│   └── orders.ejs
├── public/
│   └── css/style.css
├── uploads/                  # Thư mục lưu ảnh tĩnh
├── seeds/
│   └── seedData.js           # File nạp dữ liệu Đà Nẵng mẫu
├── .env.example
├── server.js
└── package.json
```

5.2. File `package.json` Chuẩn

```
{
  "name": "sdn302-freshbite-backend",
  "version": "1.0.0",
  "description": "FreshBite Da Nang Coffee & Bakery REST API",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "seed": "node seeds/seedData.js"
  },
  "dependencies": {
    "@google/genai": "^0.1.1",
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "ejs": "^3.1.9",
    "express": "^4.19.2",
    "helmet": "^7.1.0",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.3.1",
    "morgan": "^1.10.0",
    "multer": "^1.4.5-lts.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

5.3. Trích đoạn Code Controller Trọng Tâm (Order & Population)

File `controllers/orderController.js`:

```
const Order = require('../models/Order');

// Tạo đơn hàng Pick-up mới
exports.createOrder = async (req, res, next) => {
  try {
    const { branchId, items, pickupTime, paymentMethod } = req.body;
    if (!items || items.length === 0) {
      return res.status(400).json({ success: false, message: 'Giỏ hàng không được rỗng!' });
    }

    const subtotal = items.reduce((sum, item) => sum + (item.price * item.quantity), 0);
    const totalAmount = subtotal; // Có thể trừ discount nếu có mã voucher
    const orderCode = 'FB-' + Math.floor(10000 + Math.random() * 90000);

    const order = await Order.create({
      orderCode,
      userId: req.user._id,
      branchId,
      items,
      subtotal,
      totalAmount,
      pickupTime,
      paymentMethod
    });

    res.status(201).json({ success: true, data: order });
  } catch (error) {
    next(error);
  }
};

// Lấy chi tiết đơn hàng (Dùng Mongoose Population tối đa điểm)
exports.getOrderById = async (req, res, next) => {
  try {
    const order = await Order.findById(req.params.id)
      .populate('userId', 'fullName email phone')
      .populate('branchId', 'name address phone')
      .populate('items.productId', 'name price imageUrl calories');

    if (!order) {
      return res.status(404).json({ success: false, message: 'Không tìm thấy đơn hàng!' });
    }
    res.json({ success: true, data: order });
  } catch (error) {
    next(error);
  }
};
```

PHẦN 6: BẢN HOÀN THIỆN MÔN 2 – MMA301 (REACT NATIVE EXPO)

6.1. Cấu trúc Source Code

```
mma301-freshbite-expo/
├── assets/
│   ├── adaptive-icon.png
│   ├── icon.png
│   └── splash.png
├── src/
│   ├── components/
│   │   ├── CustomButton.js
│   │   ├── CustomInput.js
│   │   ├── ProductCard.js
│   │   ├── BranchCard.js
│   │   └── OrderStatusBadge.js
│   ├── constants/
│   │   ├── Colors.js         # Tone nâu cà phê, be bánh ngọt, xanh lá
│   │   └── Config.js         # API_BASE_URL (http://10.0.2.2:5000/api)
│   ├── context/
│   │   ├── AuthContext.js    # Lưu user và token vào AsyncStorage
│   │   └── CartContext.js    # Quản lý mảng sản phẩm giỏ hàng
│   ├── navigation/
│   │   ├── AppNavigator.js
│   │   ├── BottomTabNavigator.js  # Tabs: Home, Map, Orders, Profile
│   │   └── MainStackNavigator.js  # Screens: Detail, Cart, Checkout, AIChat
│   ├── screens/
│   │   ├── auth/
│   │   │   ├── LoginScreen.js
│   │   │   └── RegisterScreen.js
│   │   ├── home/
│   │   │   ├── HomeScreen.js
│   │   │   └── ProductDetailScreen.js
│   │   ├── map/
│   │   │   └── StoreLocatorScreen.js    # react-native-maps + expo-location
│   │   ├── cart/
│   │   │   ├── CartScreen.js
│   │   │   └── CheckoutPickupScreen.js
│   │   ├── orders/
│   │   │   ├── OrderHistoryScreen.js
│   │   │   └── OrderDetailScreen.js
│   │   ├── ai/
│   │   │   └── GeminiChatScreen.js      # Chat tư vấn món ăn (Slide 32)
│   │   └── profile/
│   │       └── ProfileScreen.js         # Đổi avatar qua expo-image-picker
│   └── services/
│       ├── api.js                       # Axios Client đính kèm token
│       └── notificationService.js
├── App.js
├── app.json
└── package.json
```

6.2. Dependencies Cần Thiết (`package.json`)

```
{
  "dependencies": {
    "@react-navigation/bottom-tabs": "^6.5.20",
    "@react-navigation/native": "^6.1.17",
    "@react-navigation/native-stack": "^6.9.26",
    "axios": "^1.6.8",
    "expo": "~51.0.0",
    "expo-image-picker": "~15.0.5",
    "expo-location": "~17.0.1",
    "expo-notifications": "~0.28.1",
    "expo-status-bar": "~1.12.1",
    "react": "18.2.0",
    "react-native": "0.74.1",
    "react-native-maps": "1.14.0",
    "react-native-safe-area-context": "4.10.1",
    "react-native-screens": "~3.31.1",
    "@react-native-async-storage/async-storage": "1.23.1"
  }
}
```

6.3. Chi tiết Màn hình Bản đồ Đà Nẵng (`StoreLocatorScreen.js`)

Trích đoạn màn hình Google Maps lấy trọn điểm Slide 32:

```jsx
import React, { useState, useEffect } from 'react';
import { StyleSheet, View, Text, TouchableOpacity, FlatList } from 'react-native';
import MapView, { Marker } from 'react-native-maps';
import * as Location from 'expo-location';
import axios from 'axios';
import { API_BASE_URL } from '../../constants/Config';

export default function StoreLocatorScreen({ navigation }) {
  const [branches, setBranches] = useState([]);
  const [region, setRegion] = useState({
    latitude: 16.0544, // Tọa độ trung tâm TP. Đà Nẵng
    longitude: 108.2022,
    latitudeDelta: 0.08,
    longitudeDelta: 0.08,
  });
  const [selectedBranch, setSelectedBranch] = useState(null);

  useEffect(() => {
    (async () => {
      let { status } = await Location.requestForegroundPermissionsAsync();
      if (status === 'granted') {
        let location = await Location.getCurrentPositionAsync({});
        // Cập nhật vị trí người dùng tại Đà Nẵng
      }
      const res = await axios.get(`${API_BASE_URL}/branches`);
      setBranches(res.data.data);
      if (res.data.data.length > 0) setSelectedBranch(res.data.data[0]);
    })();
  }, []);

  return (
    <View style={styles.container}>
      <MapView style={styles.map} region={region}>
        {branches.map((branch) => (
          <Marker
            key={branch._id}
            coordinate={{ latitude: branch.latitude, longitude: branch.longitude }}
            title={branch.name}
            description={branch.address}
            pinColor="#6F4E37"
            onPress={() => setSelectedBranch(branch)}
          />
        ))}
      </MapView>
      {selectedBranch && (
        <View style={styles.card}>
          <Text style={styles.title}>{selectedBranch.name}</Text>
          <Text style={styles.address}>{selectedBranch.address}</Text>
          <Text style={styles.hours}>Giờ mở cửa: {selectedBranch.openingHours}</Text>
          <TouchableOpacity
            style={styles.button}
            onPress={() => navigation.navigate('Home', { branchId: selectedBranch._id })}
          >
            <Text style={styles.btnText}>Chọn Chi Nhánh Này Đặt Món</Text>
          </TouchableOpacity>
        </View>
      )}
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1 },
  map: { flex: 1 },
  card: { position: 'absolute', bottom: 20, left: 16, right: 16, backgroundColor: '#FFF', padding: 16, bord
erRadius: 12, elevation: 5 },
  title: { fontSize: 16, fontWeight: 'bold', color: '#6F4E37' },
  address: { fontSize: 13, color: '#666', marginTop: 4 },
  hours: { fontSize: 12, color: '#888', marginTop: 2 },
  button: { marginTop: 12, backgroundColor: '#6F4E37', paddingVertical: 10, borderRadius: 8, alignItems: 'c
enter' },
  btnText: { color: '#FFF', fontWeight: 'bold' }
});
```

PHẦN 7: BẢN HOÀN THIỆN MÔN 3 – PRM393 (FLUTTER & DART)

7.1. Cấu trúc Source Code Chuẩn Layered / MVVM

```
prm393_freshbite_flutter/
├── android/
├── ios/
├── lib/
│   ├── models/
│   │   ├── user_model.dart
│   │   ├── branch_model.dart
│   │   ├── product_model.dart
│   │   └── order_model.dart
│   ├── providers/
│   │   ├── auth_provider.dart
│   │   ├── branch_provider.dart
│   │   └── cart_provider.dart
│   ├── services/
│   │   ├── api_service.dart
│   │   ├── notification_service.dart
│   │   └── gemini_service.dart
│   ├── utils/
│   │   ├── app_colors.dart
│   │   ├── app_theme.dart
│   │   └── constants.dart
│   ├── views/
│   │   ├── screens/
│   │   │   ├── auth/
│   │   │   │   ├── login_screen.dart
│   │   │   │   └── register_screen.dart
│   │   │   ├── home/
│   │   │   │   ├── home_screen.dart
│   │   │   │   └── product_detail_screen.dart
│   │   │   ├── map/
│   │   │   │   └── store_map_screen.dart       # google_maps_flutter
│   │   │   ├── cart/
│   │   │   │   ├── cart_screen.dart
│   │   │   │   └── checkout_pickup_screen.dart
│   │   │   ├── orders/
│   │   │   │   ├── order_history_screen.dart
│   │   │   │   └── order_detail_screen.dart
│   │   │   ├── ai/
│   │   │   │   └── ai_barista_screen.dart      # Gemini Chat UI
│   │   │   └── profile/
│   │   │       └── profile_screen.dart
│   │   └── widgets/
│   │       ├── product_card.dart
│   │       ├── custom_text_field.dart
│   │       └── order_status_stepper.dart
│   └── main.dart
├── test/
│   ├── unit/
│   │   └── cart_provider_test.dart             # Unit Test bắt buộc (Rubric)
│   └── widget/
│       └── login_screen_test.dart              # Widget Test bắt buộc (Rubric)
└── pubspec.yaml
```

7.2. File `pubspec.yaml` Đầy Đủ

```yaml
name: prm393_freshbite_flutter
description: FreshBite Da Nang - Order Ahead and Pickup Mobile App
publish_to: 'none'
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.6
  provider: ^6.1.2
  http: ^1.2.1
  shared_preferences: ^2.2.3
  google_maps_flutter: ^2.6.0
  geolocator: ^11.0.0
  image_picker: ^1.1.0
  flutter_local_notifications: ^17.1.0
  intl: ^0.19.0

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
  assets:
    - assets/images/
```

7.3. Code Unit Test Bắt Buộc Theo Rubric (`test/unit/cart_provider_test.dart`)

Giảng viên PRM393 yêu cầu bắt buộc tối thiểu 1 Unit Test:

```dart
import 'package:flutter_test/flutter_test.dart';
import 'package:prm393_freshbite_flutter/models/product_model.dart';
import 'package:prm393_freshbite_flutter/providers/cart_provider.dart';

void main() {
  group('Kiểm thử Đơn vị (Unit Test) - CartProvider Logic', () {
    late CartProvider cartProvider;

    setUp(() {
      cartProvider = CartProvider();
    });

    test('Giỏ hàng ban đầu phải có số lượng là 0 và tổng tiền bằng 0', () {
      expect(cartProvider.itemCount, 0);
      expect(cartProvider.totalAmount, 0.0);
    });

    test('Thêm 1 sản phẩm vào giỏ hàng và kiểm tra tính toán tổng tiền', () {
      final product = ProductModel(
        id: 'p1',
        name: 'Cà phê Muối Đà Nẵng',
        price: 32000,
        imageUrl: 'https://example.com/caphemuoi.png',
        calories: 180,
      );

      cartProvider.addItem(product, quantity: 2);

      expect(cartProvider.itemCount, 1);
      expect(cartProvider.totalItemsCount, 2);
      expect(cartProvider.totalAmount, 64000.0);
    });

    test('Tăng số lượng sản phẩm và kiểm tra tổng tiền tự cập nhật', () {
      final product = ProductModel(
        id: 'p1',
        name: 'Cà phê Muối Đà Nẵng',
        price: 32000,
        imageUrl: 'https://example.com/caphemuoi.png',
        calories: 180,
      );

      cartProvider.addItem(product, quantity: 1);
      cartProvider.updateQuantity('p1', 3);

      expect(cartProvider.totalAmount, 96000.0);
    });

    test('Xóa sản phẩm khỏi giỏ hàng', () {
      final product = ProductModel(
        id: 'p1',
        name: 'Cà phê Muối Đà Nẵng',
        price: 32000,
        imageUrl: 'https://example.com/caphemuoi.png',
        calories: 180,
      );

      cartProvider.addItem(product, quantity: 1);
      cartProvider.removeItem('p1');

      expect(cartProvider.itemCount, 0);
```

```
      expect(cartProvider.totalAmount, 0.0);
    });
  });
}
```

7.4. Code Widget Test Bắt Buộc Theo Rubric (`test/widget/login_screen_test.dart`)

Giảng viên PRM393 yêu cầu bắt buộc tối thiểu 1 Widget Test:

```dart
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:provider/provider.dart';
import 'package:prm393_freshbite_flutter/views/screens/auth/login_screen.dart';
import 'package:prm393_freshbite_flutter/providers/auth_provider.dart';

void main() {
  testWidgets('Kiểm thử giao diện Đăng nhập và validate form', (WidgetTester tester) async {
    // Khởi tạo màn hình trong cây Widget với ChangeNotifierProvider
    await tester.pumpWidget(
      MultiProvider(
        providers: [
          ChangeNotifierProvider(create: (_) => AuthProvider()),
        ],
        child: const MaterialApp(
          home: LoginScreen(),
        ),
      ),
    );

    // 1. Kiểm tra tồn tại trường Email và Mật khẩu
    expect(find.byType(TextFormField), findsNWidgets(2));
    expect(find.text('Đăng nhập'), findsOneWidget);

    // 2. Nhập sai định dạng Email
    await tester.enterText(find.byType(TextFormField).first, 'emailsai');
    await tester.enterText(find.byType(TextFormField).last, '123'); // mật khẩu ngắn

    // 3. Bấm nút đăng nhập
    await tester.tap(find.widgetWithText(ElevatedButton, 'Đăng nhập'));
    await tester.pump();

    // 4. Kiểm tra xem thông báo lỗi validation có hiển thị không
    expect(find.text('Email không đúng định dạng hợp lệ!'), findsOneWidget);
    expect(find.text('Mật khẩu phải chứa ít nhất 6 ký tự!'), findsOneWidget);
  });
}
```

PHẦN 8: BẢNG PHÂN CHIA CÔNG VIỆC CHI TIẾT 4 TUẦN (CHUẨN 20% MỖI NGƯỜI)

8.1. Nhóm SDN302 & MMA301 (5 Thành viên)

Thành viên	Năng lực & Vai trò	Trách nhiệm tại SDN302 (Backend) [20%]	Trách nhiệm tại MMA301 (React Native) [20%]
Nguyễn Chí Vương	FE Lead (Dùng AI hỗ trợ BE)	- Viết tài liệu Postman Collection đầy đủ các kịch bản test - Viết API Authentication (`/api/auth/register` , `/api/auth/login` , `/api/auth/me`)	- Khởi tạo Expo, cấu hình React Navigation (Tabs & Stacks) - Code Màn hình: Home, Danh mục bánh/nước, Chi tiết món - Xây dựng `CartContext` quản lý giỏ hàng
Lê Đức Sơn	FE Lead (Dùng AI hỗ trợ BE)	- Viết API Feedback/Review món ăn - Viết Proxy Endpoint kết nối Google Gemini AI (`/api/ai/chat`)	- Code Màn hình: Giỏ hàng (Cart) & Form Đặt hẹn giờ lấy (Pickup Form) - Tích hợp Google Maps (`react-native-maps`) hiển thị 4 chi nhánh ĐN - Tích hợp `expo-image-picker` đổi avatar
Nguyễn Xuân Thịnh	BE Core Lead	- Khởi tạo Server Express, kết nối MongoDB Atlas - Viết Core API: CRUD Products, Categories & Orders - Xử lý Mongoose Population và lọc danh mục	- Viết Axios Service Client kết nối Backend - Code Màn hình: Lịch sử đơn hàng & Chi tiết tiến trình đơn (Stepper) - Hỗ trợ tích hợp dữ liệu giữa BE và FE
Nguyễn Đắc Thịnh	BE Beginner (Dùng AI hỗ trợ)	- Viết API Chi nhánh (`/api/branches`) gồm 4 chi nhánh Đà Nẵng - Viết Middleware bắt lỗi toàn cục (`errorMiddleware.js`) & Logger	- Code Màn hình: Login, Register, Profile tài khoản - Xử lý lưu session & JWT Token vào `AsyncStorage` - Viết Báo cáo kỹ thuật phần State Flow & Database Design
Lê Tự Nhật Thạch	BE Beginner (Dùng AI hỗ trợ)	- Cấu hình Multer upload ảnh món ăn - Viết file script `seedData.js` nạp sẵn 4 chi nhánh và 20 món ăn mẫu Đà Nẵng - Dựng trang EJS Admin Dashboard xem doanh thu (Slot 7)	- Code Màn hình: Chatbot Gemini AI tư vấn món ăn (Slide 32) - Cấu hình `expo-notifications` nhận thông báo đơn hàng - Đóng gói file `.apk` và hoàn thiện Slide thuyết trình

---

8.2. Nhóm PRM393 (5 Thành viên)

Thành viên	Tình trạng & Năng lực	Trọng tâm công việc tại PRM393 (Flutter)	Sản phẩm bàn giao cụ thể [Đúng 20%]
Nguyễn Chí Vương	Học cả 3 môn (FE)	**UI Core & Navigation**	- Dựng khung sườn dự án, Asset ảnh, Font, Color Theme cà phê - Code UI Màn hình Home (Banner slider, Category tabs, Product grid) - Code UI Màn hình Chi tiết món (Product Detail)
Lê Đức Sơn	Học cả 3 môn (FE)	**UI Cart, Checkout & Maps**	- Code UI Màn hình Giỏ hàng (Cart Screen) & Đặt hẹn lấy hàng (Checkout Screen) - Tích hợp `google_maps_flutter` hiển thị Marker 4 chi nhánh Đà Nẵng - Tích hợp `image_picker` đổi ảnh đại diện cá nhân
Nguyễn Đắc Thịnh	Học cả 3 môn (BE dùng AI)	**Auth UI & Technical Report**	- Code UI Màn hình Login & Register có Validate Regex theo Lab 7 - Code UI Màn hình Lịch sử đơn hàng & Chi tiết mã đơn FB-XXXX - Soạn thảo trọn vẹn file `Technical Report.pdf` theo 7 mục của trường
Cao Nguyễn Đăng Khoa	Đã học SDN/MMA, đi làm (BE)	**Data Layer & State Logic**	- Viết toàn bộ Data Models Dart (`fromJson` , `toJson` theo Lab 8) - Viết `ApiService` kết nối gọi Backend Node.js - Viết `CartProvider` xử lý toàn bộ logic tính tiền giỏ hàng - **Viết Unit Test** kiểm thử giỏ hàng (`cart_provider_test.dart`)
Hồ Trung Hiếu	Đã học SDN/MMA, đi làm (BE)	**AI Service & Testing/Build**	- Tích hợp Gemini AI Service (Màn hình chat tư vấn bánh & nước) - Cấu hình `flutter_local_notifications` báo món đã làm xong - **Viết Widget Test** kiểm thử Form đăng nhập (`login_screen_test.dart`) - Tối ưu và chạy lệnh `flutter build apk --release` xuất file APK

PHẦN 9: CHECKLIST CHUẨN BỊ BẢO VỆ ĐỒ ÁN (LẤY ĐIỂM 10)

1. **Điểm nhấn Đà Nẵng:** Khi mở app lên, Google Maps định vị ngay tại TP. Đà Nẵng với 4 chi nhánh rõ ràng (đặc biệt là chi nhánh FPT City). Giảng viên sẽ thấy ngay sự đầu tư nghiêm túc và thực tế.
2. **Kịch bản Demo không dính lỗi Shipper:**
3. Đặt món $\rightarrow$ Chọn chi nhánh FPT City $\rightarrow$ Hẹn 15 phút ghé lấy.
4. Mở màn hình Admin (Web EJS hoặc In-App) $\rightarrow$ Barista bấm "Món đã sẵn sàng".
5. App khách lập tức nhảy thông báo $\rightarrow$ Khách đến đọc mã đơn.
6. **Showcase Kỹ thuật tại PRM393:**
7. Mở terminal gõ: `flutter test` $\rightarrow$ Tất cả test case Unit Test và Widget Test đều Passed xanh lét.
8. Mở thư mục `build/app/outputs/flutter-apk/app-release.apk` chứng minh đã build xong APK Release.
9. **Showcase Kỹ thuật tại SDN302:**
10. Trình chiếu file Postman Collection test đầy đủ các mã HTTP (200, 201, 400, 401, 404, 500).
11. Mở MongoDB Compass cho thấy các quan hệ bảng `Order` được populate chi tiết `User`, `Branch`, `Product`.